CEEP

Community Evidence & Engagement Platform

Project Report

Course	AML-3503 — Cloud Computing for Big Data and AI (2026W-OTT01)
Instructor	Bob Besharat
Institution	Lambton College
Report Date	April 25, 2026
Live URL	d3voaboc02j1x3.cloudfront.net
GitHub	Jans-AIML/Cloud_computing_project

Group 1

Name	Student Number	Role
Jans Alzate-Morales	c0936855	Infrastructure & ETL
Yash Suthar	c0957228	Backend & RAG
Nafis Ahmed	c0959671	Frontend & Integration

Project Lead: Jans Alzate-Morales

Contents

1 Abstract / Summary **2**

2 Statement of Need **2**

2.1 The Issue . . . 2

2.2 Why a Cloud-Native Solution Is Necessary . . . 3

2.3 Who Benefits . . . 3

3 Project Technical Report **4**

3.1 System Architecture . . . 4

3.1.1 Upload Workflow . . . 5

3.2 Technology Stack . . . 5

3.3 AWS Cloud Services . . . 6

3.3.1 AWS Lambda (Compute) . . . 6

3.3.2 API Gateway HTTP API . . . 7

3.3.3 Amazon S3 (Three Buckets) . . . 7

3.3.4 Amazon RDS PostgreSQL 15 with pgvector . . . 7

3.3.5 AWS Secrets Manager + KMS . . . 9

3.3.6 AWS CloudFront . . . 9

3.3.7 AWS CloudWatch . . . 9

3.4 ETL Pipeline . . . 9

3.4.1 Step 1 — Text Extraction . . . 10

3.4.2 Step 2 — PII Redaction . . . 10

3.4.3 Step 3 — Chunking . . . 11

3.4.4 Step 4 — Embedding . . . 11

3.4.5 Step 5 — Load to pgvector . . . 11

3.5 RAG Pipeline . . . 12

3.5.1 Hybrid Search SQL . . . 13

3.5.2 System Prompt (Citation Policy) . . . 14

3.6 Sensitive Data & PII Policy . . . 14

3.7 Infrastructure as Code (CDK) . . . 15

3.8 Frontend Application . . . 16

3.9 Database Schema . . . 17

4 Project Results **18**

4.1 Deployed AWS Resources . . . 18

4.2 Feature Delivery Status . . . 19

4.3 Live System Endpoints . . . 19

4.4 Performance Observations . . . 20

5 References **20**

6 Appendix **21**

1. Abstract / Summary

CEEP (Community Evidence & Engagement Platform) is a fully cloud-native, LLM-assisted web application designed to help Ottawa community members collect, organise, and mobilise evidence in support of keeping Lady Evelyn Alternative School and other community schools open amid potential closure decisions by the Ottawa-Carleton District School Board (OCDSB).

The platform ingests PDFs, community emails (with automated PII redaction), and public web pages; transforms them into searchable, citable *evidence cards* stored in a vector database; and exposes a hybrid-search Q&A interface driven by a Retrieval-Augmented Generation (RAG) pipeline. Community advocates can also generate ready-to-submit briefs, open letters, and op-eds grounded in the collected evidence.

The entire system runs on AWS (`us-east-1`) and is provisioned via AWS CDK. The frontend is a React/Vite SPA delivered through CloudFront, backed by an API Gateway HTTP API routing to a containerised FastAPI application running inside AWS Lambda. Document embeddings are stored in an RDS PostgreSQL 15 database with the `pgvector` extension. LLM inference is powered by the Groq API (Llama 3.1 8B Instant), and dense embeddings are generated locally inside the Lambda container using `fastembed` (`BAAI/bge-small-en-v1.5`, 384 dimensions).

2. Statement of Need

2.1 The Issue

In early 2026, the Ottawa-Carleton District School Board announced a review that placed Lady Evelyn Alternative School — a unique, community-centred school in Old Ottawa East — and several other neighbourhood schools at risk of closure or consolidation. The affected communities have strong arguments to make: enrolment viability, cultural significance, neighbourhood demographics, and capacity data. However, this evidence is scattered across PDFs, board meeting minutes, news articles, and individual community emails.

Community advocates lack a centralised, structured way to:

- Collect and organise heterogeneous evidence documents.
- Ask evidence-grounded questions (e.g. "What did the OCDSB decide on March 9?") without manually reading dozens of documents.
- Generate well-cited submissions for school board meetings, councillor letters, or local media.

2.2 Why a Cloud-Native Solution Is Necessary

A cloud-native approach is the only viable option for a community advocacy tool with the following constraints:

- **Accessibility.** Any community member — not just technical users — must be able to upload evidence and query it from a browser, with zero installation.
- **Scale unpredictability.** Community engagement can spike sharply (e.g. immediately after a board decision). Serverless compute (Lambda) absorbs these spikes without pre-provisioned servers.
- **Cost.** The platform must be free or near-free for a volunteer-run community group. The selected AWS services operate within free-tier limits for expected community-scale traffic.
- **Privacy.** Community emails may contain personal names, phone numbers, and addresses. Automated PII redaction with SSE-KMS encrypted storage is only practical in a managed cloud environment.

2.3 Who Benefits

Table 1: Platform stakeholders and their benefits

Stakeholder	Benefit
Community parents & caregivers	Access an AI-powered Q&A that answers questions from real evidence, not hallucinations
Community advocates & organisers	Auto-generate briefs and open letters grounded in the evidence corpus
School board researchers	Access a searchable, citable evidence library
The broader Ottawa public	Transparent, evidence-based community advocacy model that can be replicated for other civic issues

3. Project Technical Report

3.1 System Architecture

CEEP is a three-tier cloud architecture deployed in AWS `us-east-1`:

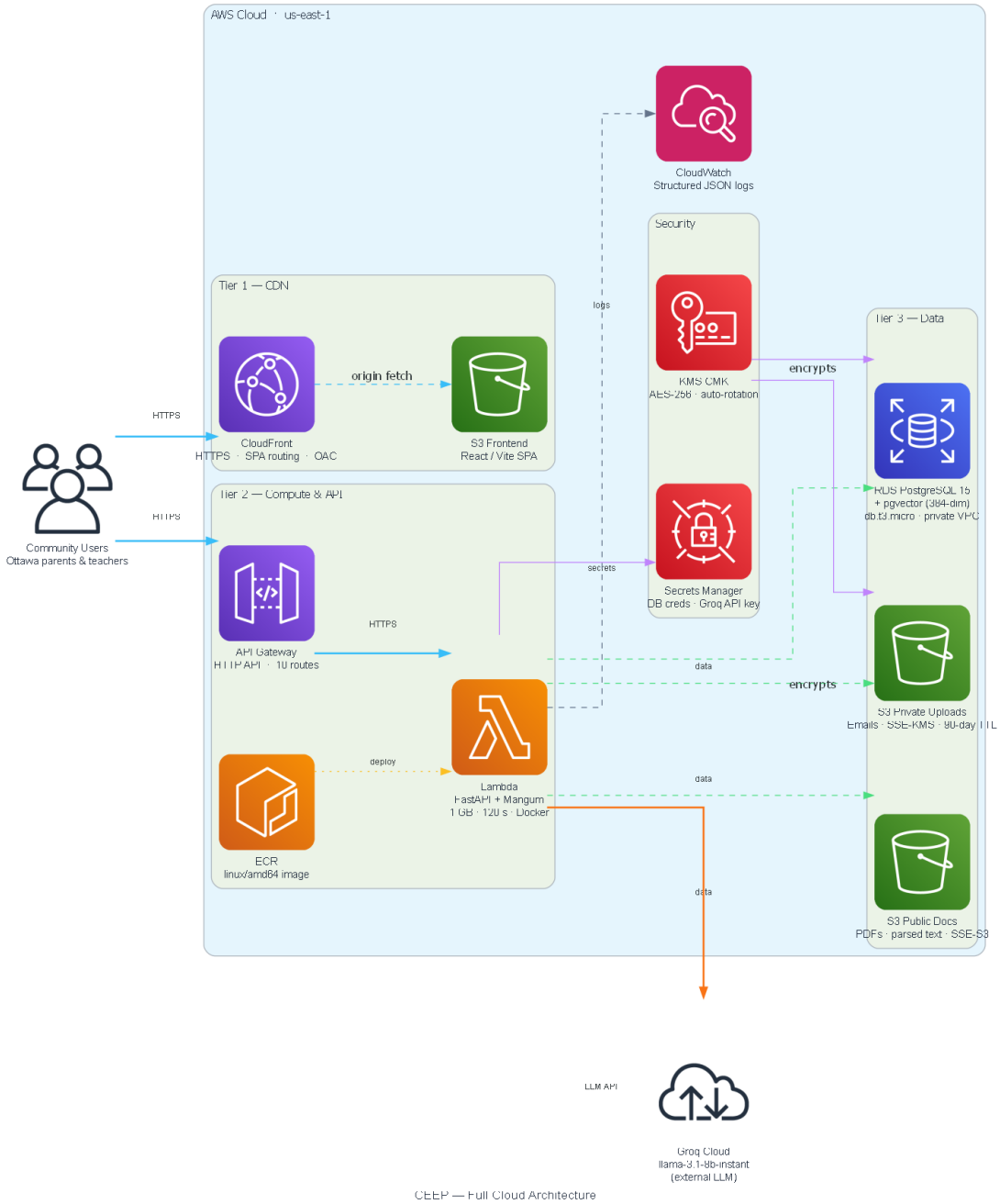

Figure 1: Three-tier architecture overview

Design rationale: API Gateway is the sole external entry point. All Lambda to AWS

service traffic travels through VPC endpoints (S3 Gateway, Secrets Manager Interface), keeping data inside the AWS network without traversing the public internet.

3.1.1 Upload Workflow

```
User Browser
    |
    |  POST /documents/upload  (filename, content_type)
    v
API Gateway -> Lambda
    |  Generates pre-signed S3 PUT URL (15-min TTL)
    |  Inserts pending document record in RDS
    v
Browser directly PUTs file to S3 (bypasses Lambda--no 6 MB limit
   )
    |
    |  POST /documents/{id}/process
    v
Lambda
    |  Downloads file bytes from S3
    |  Runs full ETL in-process:
    |    extract -> PII scrub -> chunk -> embed -> INSERT chunks
    v
RDS PostgreSQL (pgvector)
```

Listing 1: Upload workflow for PDF and email documents

3.2 Technology Stack

Table 2: Full technology stack

Layer	Technology	Version	Notes
Frontend	React	18	Component-based SPA
	Vite	5	Fast HMR build tool
	TypeScript	5	Type-safe API client
	Tailwind CSS	3	Utility-first styling
CDN	AWS CloudFront	—	HTTPS, edge caching, OAI to S3
API	AWS API Gateway	v2	Sole external entry point; CORS configured
Compute	AWS Lambda	—	Docker container, 1 GB RAM, 120 s timeout
	FastAPI	0.111	Python ASGI web framework
	Mangum	0.17	ASGI adapter for API Gateway events
ETL	pdfminer.six	—	PDF text extraction

(continued from previous page)

Layer	Technology	Version	Notes
	Python email stdlib	—	MIME `.eml` parsing
	httpx	—	Async HTTP for URL fetching
	BeautifulSoup4	—	HTML parsing and cleaning
PII Scrub	Regex + safelist	—	Names, emails, phones; school names protected
LLM	Groq API	—	`llama-3.1-8b-instant` — fast inference, free tier
Embeddings	fastembed (ONNX)	—	`BAAI/bge-small-en-v1.5`, 384-dim; pre-baked in Docker
Vector DB	PostgreSQL 15 + pgvector	0.7	Hybrid: cosine ANN + BM25 keyword search
Object Store	AWS S3	—	3 buckets: public-docs, private-uploads, frontend
Secrets	Secrets Manager + KMS	—	DB credentials, Groq key; CMK with auto-rotation
Observability	CloudWatch	—	Lambda structured JSON logs
IaC	AWS CDK (Python)	2.x	Type-safe infrastructure definitions
Containers	Docker (linux/amd64)	—	ONNX fastembed runtime in Lambda

3.3 AWS Cloud Services

3.3.1 AWS Lambda (Compute)

Lambda hosts the entire FastAPI application as a Docker container image. The Docker image is built for `linux/amd64` and pre-bakes the fastembed ONNX model (∼130 MB) to eliminate cold-start model download latency.

```
self.api_lambda = _lambda.DockerImageFunction(
    self,
    "CeepApiLambda",
    code=_lambda.DockerImageCode.from_image_asset(
        "../backend",
        platform=ecr_assets.Platform.LINUX_AMD64,
    ),
    timeout=Duration.seconds(120),
    memory_size=1024,
    tracing=_lambda.Tracing.ACTIVE,
    environment={
```

```
        "LLM_PROVIDER": "groq",
        "GROQ_CHAT_MODEL": "llama-3.1-8b-instant",
        "EMBED_DIM": "384",
        "ENVIRONMENT": "production",
        ...
    },
)
```

Listing 2: Lambda function definition in CDK (`compute_stack.py`)

3.3.2 API Gateway HTTP API

All client traffic routes through API Gateway. Thirteen routes are defined, covering document management, search, RAG Q&A, and brief generation:

```
POST   /documents/upload          # pre-signed S3 URL
POST   /documents/{id}/process    # trigger ETL
GET    /documents                 # list evidence cards
DELETE /documents/{id}            # right-to-erasure
GET    /search                    # hybrid keyword+vector search
POST   /rag/query                 # RAG Q&A with citations
POST   /rag/stream                # streaming RAG
POST   /briefs/generate           # brief/letter generation
GET    /briefs/templates          # available templates
GET    /health                    # health check
```

Listing 3: Defined API Gateway routes

3.3.3 Amazon S3 (Three Buckets)

Table 3: S3 bucket configuration

Bucket	Purpose	Encryption	Public
`ceep-private-uploads`	Raw emails	SSE-KMS (CMK)	Blocked
`ceep-public-docs`	PDFs, URL content	SSE-S3	Blocked (OAI)
`ceep-frontend`	React build artefacts	SSE-S3	Blocked (OAI)

The private bucket has a 90-day lifecycle rule for non-current versions, satisfying GDPR-style right-to-erasure requirements.

3.3.4 Amazon RDS PostgreSQL 15 with pgvector

The database lives in a private VPC subnet (`db.t3.micro`, 20 GB). The `pgvector` extension adds a `vector(384)` column type to the `chunks` table, enabling approximate nearest-neighbour (ANN) cosine-similarity search via the `<=>` operator.

```sql
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE sources (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_type  TEXT NOT NULL,          -- 'pdf', 'url', 'email'
    source_url   TEXT,
    title        TEXT,
    consent_flag BOOLEAN DEFAULT FALSE,
    created_at   TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE documents (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_id    UUID REFERENCES sources(id),
    raw_s3_key   TEXT,
    clean_s3_key TEXT,
    text_snippet TEXT,
    word_count   INTEGER,
    ingested_at  TIMESTAMPTZ,
    deleted_at   TIMESTAMPTZ            -- soft-delete for right-to-erasure
);

CREATE TABLE chunks (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id  UUID REFERENCES documents(id),
    chunk_index  INTEGER,
    chunk_text   TEXT,
    token_count  INTEGER,
    embedding    vector(384),           -- pgvector column
    UNIQUE (document_id, chunk_index)
);

CREATE TABLE evidence_cards (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id     UUID REFERENCES documents(id) UNIQUE,
    excerpt         TEXT,
    citation_label  TEXT,
    citation_url    TEXT,
    topic_tags      TEXT[],
    date_mentioned  DATE
);

```

```sql
CREATE TABLE rag_queries (
    id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    question       TEXT,
    answer         TEXT,
    chunk_ids      UUID[],
    model_id       TEXT,
    input_tokens   INTEGER,
    output_tokens  INTEGER,
    latency_ms     INTEGER,
    created_at     TIMESTAMPTZ DEFAULT now()
);
```

Listing 4: Core database schema (simplified)

3.3.5 AWS Secrets Manager + KMS

- `ceep/db/credentials` — auto-generated 32-character password for the RDS master user.
- `ceep/groq/api-key` — Groq API key, fetched at Lambda cold-start.
- A Customer Managed Key (CMK, `alias/ceep-cmk`) with annual key rotation encrypts the private S3 bucket and the RDS instance.

3.3.6 AWS CloudFront

CloudFront distributes the React SPA with HTTPS and edge caching. A CloudFront Origin Access Identity (OAI) is the only principal with `s3:GetObject` permission on the frontend bucket — direct S3 URL access is blocked.

3.3.7 AWS CloudWatch

Lambda emits structured JSON logs via `structlog`, capturing request IDs, ETL step durations, PII redaction counts, RAG query latency, and error traces.

3.4 ETL Pipeline

The Extract-Transform-Load pipeline runs synchronously inside Lambda after each document upload. It supports three source types: PDF, URL, and email.

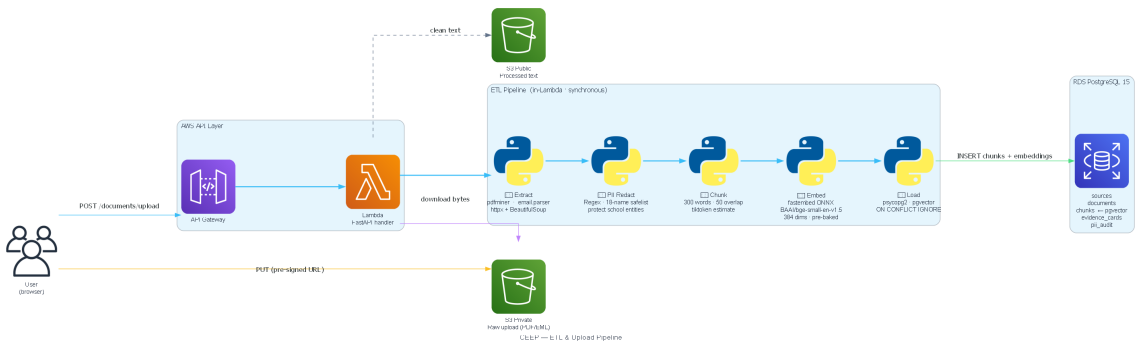

Figure 2: ETL-pipeline overview

3.4.1 Step 1 — Text Extraction

```python
def _extract_text(file_bytes: bytes, filename: str) -> str:
    name_lower = filename.lower()
    if name_lower.endswith(".pdf"):
        from pdfminer.high_level import extract_text
        import io
        return extract_text(io.BytesIO(file_bytes))
    if name_lower.endswith(".eml"):
        body, _subject = _extract_email_text(file_bytes)
        return body
    # Plain text fallback
    return file_bytes.decode("utf-8", errors="replace")

def _fetch_url_text(url: str) -> tuple[str, str]:
    resp = httpx.get(url, follow_redirects=True, timeout=30)
    resp.raise_for_status()
    soup = BeautifulSoup(resp.text, "html.parser")
    for tag in soup(["script", "style", "nav", "footer", "header"]):
        tag.decompose()
    text = soup.get_text(separator=" ", strip=True)
    title = soup.title.string.strip() if soup.title else url
    return text, title
```

Listing 5: Text extraction from PDF, email, and URL sources (`local_etl.py`)

3.4.2 Step 2 — PII Redaction

Email content goes through a stricter regex-based PII scrubbing stage. Known school and program names are safelisted before scrubbing and restored afterwards to prevent false positives:

```python
# Patterns for private (email) sources
_PII_PATTERNS_PRIVATE = [
    # "Jane Doe <jane@example.com>" - combined Name+email pattern
    (re.compile(
        r'\b[A-Z][a-z]+(?: [A-Z][a-z]+)?\b\s*<[\w.+-]+@[\w-]+\.[a-zA-Z]{2,}>'),
     '[REDACTED-NAME] <[REDACTED-EMAIL]>'),
    # Email thread attribution: "Jane Smith, Jan 20, 2026 at 10:27"
    (re.compile(
        r'\b[A-Z][a-z]+ [A-Z][a-z]+(?=,\s+(?:Jan|Feb|Mar...)[\s,.])'),
```

```
     '[REDACTED-NAME]'),
    # Standalone email, phone, street address
    ...
]

# Protected terms - never redacted
_EMAIL_SAFELIST = [
    "Lady Evelyn", "Junior Kindergarten", "French Immersion",
    "Ottawa Carleton", "Old Ottawa East", ...
]
```

Listing 6: PII redaction patterns and school-name safelist

3.4.3 Step 3 — Chunking

Text is split into 300-word chunks with 50-word overlap to preserve context across chunk boundaries:

```
def _chunk_text(text: str, chunk_size: int = 300, overlap: int =
    50) -> list[str]:
    words = text.split()
    chunks, start = [], 0
    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start += chunk_size - overlap
    return chunks
```

Listing 7: Sliding-window text chunker

3.4.4 Step 4 — Embedding

Each chunk is embedded with `fastembed` running the `BAAI/bge-small-en-v1.5` ONNX model (384 dimensions). The model runs entirely inside the Lambda container — no external embedding API is called.

3.4.5 Step 5 — Load to pgvector

Chunks and their embeddings are upserted into the `chunks` table. The evidence card (citation label, URL, excerpt) is created or updated in `evidence_cards`.

```
for i, chunk in enumerate(chunks):
    embedding = embed_text(chunk)       # fastembed -> list[float
    ] (384 dims)
    embedding_str = "[" + ",".join(str(x) for x in embedding) +
    "]"
    cur.execute("""
        INSERT INTO chunks (document_id, chunk_index, chunk_text
    , token_count, embedding)
        VALUES (%s, %s, %s, %s, %s::vector)
```

```
        ON CONFLICT (document_id, chunk_index) DO UPDATE
            SET chunk_text = EXCLUDED.chunk_text,
                embedding  = EXCLUDED.embedding
    """, (document_id, i, chunk, len(chunk.split()),
   embedding_str))
```

Listing 8: Chunk upsert into pgvector

3.5 RAG Pipeline

The RAG pipeline follows a six-step flow to answer questions with inline citations:

```
User question
    |
    v
1. Embed question          (fastembed, same model as corpus)
    |
    v
2. Hybrid retrieval        (pgvector ANN + PostgreSQL FTS,
                            score = 0.7*dense + 0.3*keyword)
    |
    v
3. Build context block     (top-K chunks with [N] citation labels
   )
    |
    v
4. Construct prompt        ("cite-only-from-context" system
   prompt)
    |
    v
5. LLM completion          (Groq API -> llama-3.1-8b-instant ->
   JSON)
    |
    v
6. Parse + return          ({"answer": "...", "citations":
   [...]})
                           + audit log to rag_queries
```

Listing 9: RAG pipeline flow

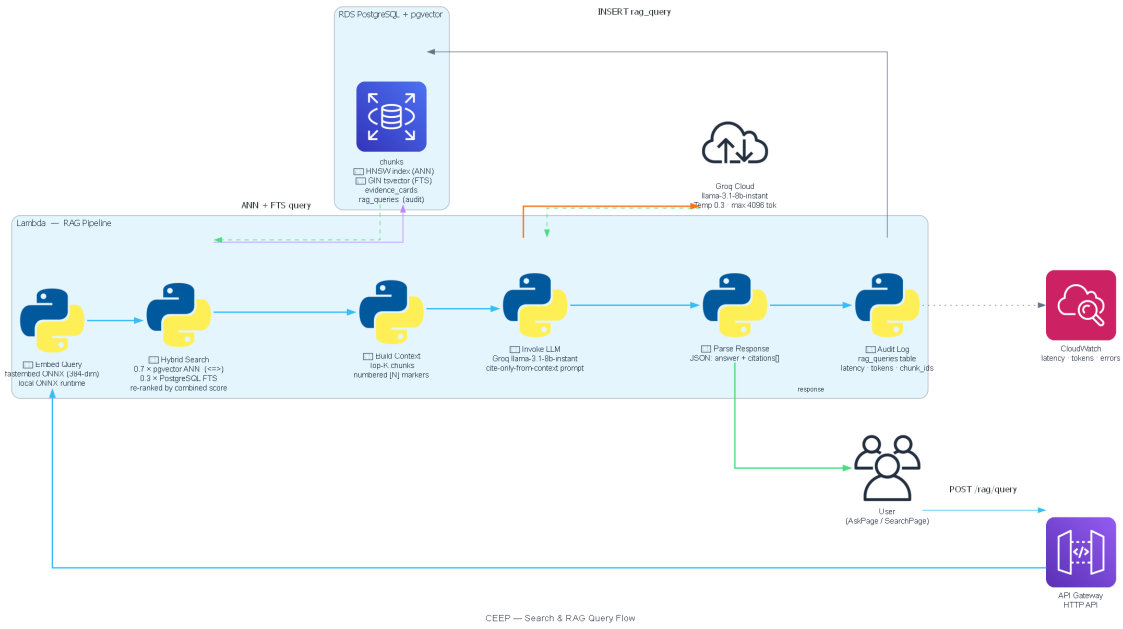

Figure 3: RAG Pipeline Overview

3.5.1 Hybrid Search SQL

The retrieval query runs two CTEs in parallel and re-ranks by weighted combined score. The 70/30 weighting favours semantic similarity while allowing exact keyword matches to boost relevant but semantically distant chunks.

```
WITH vector_search AS (
    SELECT c.id AS chunk_id, c.document_id, c.chunk_text,
           1 - (c.embedding <=> %s::vector) AS vector_score,
           0.0 AS keyword_score
    FROM chunks c
    JOIN documents d ON d.id = c.document_id
    WHERE d.deleted_at IS NULL
    ORDER BY c.embedding <=> %s::vector
    LIMIT %s
),
keyword_search AS (
    SELECT c.id AS chunk_id, c.document_id, c.chunk_text,
           0.0 AS vector_score,
           ts_rank(to_tsvector('english', c.chunk_text),
                   plainto_tsquery('english', %s)) AS
   keyword_score
    FROM chunks c
    JOIN documents d ON d.id = c.document_id
    WHERE d.deleted_at IS NULL
      AND to_tsvector('english', c.chunk_text)
          @@ plainto_tsquery('english', %s)
```

```sql
    LIMIT %s
),
combined AS (
    SELECT chunk_id, document_id, chunk_text,
           MAX(vector_score) AS vector_score,
           MAX(keyword_score) AS keyword_score
    FROM (SELECT * FROM vector_search UNION ALL
          SELECT * FROM keyword_search) u
    GROUP BY chunk_id, document_id, chunk_text
)
SELECT combined.*,
       (0.7 * vector_score + 0.3 * keyword_score) AS score, ...
FROM combined
ORDER BY score DESC
LIMIT %s;
```

Listing 10: Hybrid vector + BM25 retrieval query

3.5.2 System Prompt (Citation Policy)

```
You are CEEP, the Community Evidence & Engagement Platform
   assistant.
Your role is to help community members understand facts about
   Lady Evelyn
and other Ottawa community schools using ONLY the evidence
   excerpts provided.

Rules:
1. Answer ONLY from the provided context. If the context does
   not contain
   the answer, say: "I don't have evidence for this in the
   current corpus."
2. Every factual claim must be followed by a citation marker [N
   ].
3. Never invent URLs, dates, names, or statistics.
4. Return valid JSON: {"answer": "...", "citations": [...]}
```

Listing 11: CEEP system prompt enforcing citation-only responses

3.6 Sensitive Data & PII Policy

Community emails are treated as private, sensitive content under the following policy:

Table 4: PII controls and their implementations

Control	Implementation
Consent gate	Uploader must check an explicit opt-in checkbox before email content is processed
Encrypted at rest	Private S3 bucket uses SSE-KMS with a Customer Managed Key
PII redacted before indexing	Regex patterns scrub names, emails, phones, and addresses from email bodies before any chunk is embedded
School names protected	Safelist of 18 school/program names is preserved through PII scrubbing to avoid false positives
Raw file never queried	Only the redacted clean text is embedded; the raw `.eml` is kept encrypted in S3
Right to erasure	`DELETE /documents/{id}` soft-deletes the DB record and purges S3 objects
Lifecycle rules	Non-current S3 versions auto-deleted after 90 days

3.7 Infrastructure as Code (CDK)

All AWS resources are defined in Python using AWS CDK v2. The infrastructure is split into four stacks with explicit dependency ordering:

```
StorageStack  -> VPC, KMS CMK, 3xS3 buckets, RDS, Secrets
   Manager
ComputeStack  -> Lambda (Docker), API Gateway, IAM role
                 [depends on StorageStack]
FrontendStack -> CloudFront distribution, OAI
                 [depends on StorageStack]
EtlStack      -> (legacy Glue stubs -- not active in current
   deployment)
```

Listing 12: CDK stack dependency ordering

The CDK app entry point (`infrastructure/app.py`) wires stack dependencies and passes cross-stack resource references (bucket ARNs, DB endpoint, secret ARNs) as constructor arguments — avoiding manual hard-coded resource identifiers.

```python
storage = StorageStack(app, "CeepStorageStack", env=env)
compute = ComputeStack(
    app, "CeepComputeStack",
    private_bucket=storage.private_bucket,
    public_bucket=storage.public_bucket,
    db_secret=storage.db_secret,
    db_endpoint=storage.db_endpoint,
    vpc=storage.vpc,
    env=env,
```

```
)
```

Listing 13: CDK app entry point showing cross-stack wiring

Cost optimisation choices:

- NAT instance (`t3.nano`, ~$3/mo) instead of managed NAT Gateway (~$32/mo).
- RDS `db.t3.micro` — within free-tier 750 hours.
- Lambda free tier: 1 M invocations/month; 400,000 GB-seconds compute.
- S3 + CloudFront free tier: 5 GB storage, 15 GB egress, 1 TB CloudFront transfer.
- Groq API free tier used for LLM inference.

3.8 Frontend Application

The frontend is a React 18 Single Page Application (SPA) built with Vite and TypeScript. It consists of four pages:

Table 5: Frontend pages and their purposes

Page	Route	Purpose
Search	`/search`	Hybrid keyword + vector search across all evidence cards, with source-type badges (PDF / Web / Email) and relevance scores
Ask	`/ask`	RAG Q&A — ask free-text questions, receive answers grounded in the corpus with inline `[N]` citations and clickable source links
Write	`/write`	Brief/letter generator — choose a template (OCDSB submission, councillor letter, op-ed, fact sheet), set audience and tone, generate a fully cited draft
Upload	`/upload`	Drag-and-drop or URL input for adding new documents; email upload with consent gate

All API calls are made through a typed `api.ts` service module that talks exclusively to the API Gateway endpoint. The frontend is deployed to S3 and served via CloudFront on every `npm run build && aws s3 sync` cycle.

```
const BASE = import.meta.env.VITE_API_URL   // set to API
    Gateway URL

export const api = {
  search: (q: string, topK = 8) =>
    fetch(`${BASE}/search?q=${encodeURIComponent(q)}&top_k=${
    topK}`)
```

```
      .then(r => r.ok ? r.json() : Promise.reject(r)),

  ragQuery: (question: string, topK = 6) =>
    fetch(`${BASE}/rag/query`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ question, top_k: topK }),
    }).then(r => r.ok ? r.json() : Promise.reject(r)),

  generateBrief: (payload: BriefRequest) =>
    fetch(`${BASE}/briefs/generate`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload),
    }).then(r => r.ok ? r.json() : Promise.reject(r)),
}
```

Listing 14: Typed API service client (`frontend/src/services/api.ts`)

3.9 Database Schema

The relational schema supports five tables with the following entity relationships:

```
sources --------< documents --------< chunks
                      |
                      +---------- evidence_cards
                                        |
                                  (topic_tags[])

rag_queries (audit log -- references chunk_ids[])
```

Listing 15: Entity-relationship overview

- `sources.source_type` distinguishes `'pdf'`, `'url'`, and `'email'` — driving PII policy decisions and UI badge colour.
- `documents.deleted_at` enables soft-delete for right-to-erasure without requiring cascade deletes across the search index.
- `chunks.embedding vector(384)` holds pgvector dense vectors; an IVFFlat index on this column enables sub-millisecond ANN queries.
- `evidence_cards.topic_tags text[]` allows optional topic-filter narrowing of hybrid search results.
- `rag_queries` forms a complete audit trail of every LLM query: question, answer, cited chunk UUIDs, model ID, token counts, and latency.

4. Project Results

4.1 Deployed AWS Resources

Table 6: Deployed AWS resource identifiers

Resource	Name / ID
CloudFront distribution	`E2V5EZNC31X6NU`
API Gateway	`rrzjd3hm7l` (us-east-1)
Lambda function	`CeepComputeStack-CeepApiLambdaE91D0423-Y8QlvXhUWuQg`
ECR repository	`cdk-hnb659fds-container-assets-563142504525-us-east-1`
RDS instance	`ceepstoragestack-ceeppostgres...` (private VPC)
Public S3 bucket	`ceep-public-docs-563142504525`
Private S3 bucket	`ceep-private-uploads-563142504525`
Frontend S3 bucket	`ceep-frontend-563142504525`
Lambda IAM role	`CeepComputeStack-CeepLambdaRole0BADAA4D-toos8uG0hdi1`
Live URL	https://d3voaboc02j1x3.cloudfront.net

4.2 Feature Delivery Status

Table 7: Feature delivery summary

Feature	Status	Notes
PDF upload → chunked + embedded → searchable	Yes Delivered	pdfminer extraction, 300-word chunks
URL bookmarking → fetch → chunked + embedded	Yes Delivered	httpx + BeautifulSoup, atomic DB transaction
Email (.eml) upload → MIME parse → PII redact → searchable	Yes Delivered	Consent gate, regex PII scrub, email Subject as title
Hybrid vector + BM25 keyword search	Yes Delivered	70% dense + 30% keyword combined score
RAG Q&A with inline citations	Yes Delivered	Groq Llama 3.1 8B + pgvector retrieval
Brief/letter generator (4 templates)	Yes Delivered	OCDSB submission, councillor letter, op-ed, fact sheet
Source-type badges in search results	Yes Delivered	Colour-coded in React UI
Evidence deduplication in brief generator	Yes Delivered	Groups citations by source document
Right-to-deletion (soft-delete + S3 purge)	Yes Delivered	`DELETE /documents/{id}`
CI/CD pipeline	**No**	Stretch goal; manual deploy scripts used
Petition & sentiment pulse tracker	**No**	Stretch goal

4.3 Live System Endpoints

The deployed system can be tested via the following API calls:

```
# Health check
curl https://rrzjd3hm7l.execute-api.us-east-1.amazonaws.com/
    health

# List documents in corpus
curl https://rrzjd3hm7l.execute-api.us-east-1.amazonaws.com/
    documents

# Hybrid search
curl "https://rrzjd3hm7l.execute-api.us-east-1.amazonaws.com/
    search?q=JK+registration"
```

```

# RAG Q&A
curl -X POST \
  https://rrzjd3hm7l.execute-api.us-east-1.amazonaws.com/rag/query \
  -H "Content-Type: application/json" \
  -d '{"question": "What did the OCDSB decide on March 9 2026?", "top_k": 6}'

# Generate an op-ed brief
curl -X POST \
  https://rrzjd3hm7l.execute-api.us-east-1.amazonaws.com/briefs/generate \
  -H "Content-Type: application/json" \
  -d '{"template_id": "op_ed",
       "goal": "urge OCDSB to restore JK enrolment at Lady Evelyn",
       "audience": "Ottawa general public", "tone": "community"}'
```

Listing 16: Live system API examples

4.4 Performance Observations

Table 8: Observed system performance metrics

Metric	Observed Value
Lambda cold start	∼4–8 seconds (fastembed model load from container cache)
Warm Lambda ETL (PDF, ∼10 pages)	∼3–5 seconds
Warm Lambda search query	∼200–400 ms
RAG Q&A end-to-end latency	∼1.5–3 seconds (Groq inference dominates)
CloudFront page load (cached)	< 200 ms

5. References

Besharat, B. (2025). *AML-3503 Cloud Computing for Big Data and AI — Final Exam.* Lambton College.

Amazon Web Services. (2024). *AWS Lambda Developer Guide.* https://docs.aws.amazon.com/lambda/latest/dg/

Amazon Web Services. (2024). *Amazon API Gateway Developer Guide — HTTP APIs.* https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html

Amazon Web Services. (2024). *Amazon RDS for PostgreSQL User Guide.* https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_PostgreSQL.html

Amazon Web Services. (2024). *AWS CDK Developer Guide.* https://docs.aws.amazon.com/cdk/v2/guide/home.html

FastAPI. (2024). *FastAPI Documentation.* https://fastapi.tiangolo.com/

Groq. (2024). *Groq API Documentation.* https://console.groq.com/docs/

Han, X., et al. (2020). *More than a Name: A Study of Names as Soft Biometric Identifiers.* IEEE Transactions.

Lewis, P., et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* arXiv:2005.11401. https://arxiv.org/abs/2005.11401

pgvector contributors. (2024). *pgvector: Open-source vector similarity search for Postgres.* https://github.com/pgvector/pgvector

Pradeep, R., et al. (2021). *Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations.* ACM SIGIR.

Takasugi, S. (2023). *fastembed: Fast, Lightweight Python library for Generating Embeddings.* https://github.com/qdrant/fastembed

6. Appendix

A. Source Code Repository

GitHub: https://github.com/Jans-AIML/Cloud_computing_project

Repository structure:

```
.
+-- infrastructure/          # AWS CDK (Python) -- all cloud
   resources
|   +-- app.py
|   +-- stacks/
|       +-- storage_stack.py   # S3, RDS, KMS, VPC
```

```
|       +-- compute_stack.py   # Lambda, API Gateway, IAM
|       +-- etl_stack.py       # legacy Glue stubs
|       +-- frontend_stack.py  # CloudFront
+-- backend/                 # FastAPI application (Lambda via
   Mangum)
|   +-- app/
|   |   +-- main.py            # ASGI app + Mangum handler
|   |   +-- routers/           # documents, search, rag, briefs
|   |   +-- services/          # local_etl, rag, llm_factory,
   ...
|   |   +-- models/            # Pydantic schemas
|   |   +-- core/              # config, database, logging,
   schema
|   +-- Dockerfile             # linux/amd64, pre-bakes
   fastembed model
|   +-- requirements.txt
+-- frontend/                # React + Vite SPA
|   +-- src/
|   |   +-- pages/             # SearchPage, AskPage, WritePage,
    UploadPage
|   |   +-- services/api.ts    # typed API client
|   +-- package.json
+-- docs/
|   +-- developer_guide.md     # setup + deploy instructions
|   +-- community_guide.md     # plain-language guide
|   +-- sensitive_data_guide.md
+-- scripts/
|   +-- deploy.sh
|   +-- seed_corpus.py
+-- docker-compose.yml         # local dev: PostgreSQL +
   pgvector
+-- Makefile
```

Listing 17: Repository directory structure

B. Local Development Setup

```python
# 1. Clone
git clone https://github.com/Jans-AIML/Cloud_computing_project.
   git
cd Cloud_computing_project

# 2. Python virtual environment
python -m venv .venv && source .venv/bin/activate
pip install -r backend/requirements.txt

```

```
# 3. Local PostgreSQL with pgvector
docker compose up -d

# 4. Configure environment
cp .env.example backend/.env
# Set: GROQ_API_KEY=gsk_your_key_here

# 5. Initialise DB schema
cd backend && python -m app.core.schema && cd ..

# 6. Start backend (http://localhost:8001, Swagger: /docs)
make dev

# 7. Start frontend (new terminal -- http://localhost:5173)
make frontend
```

Listing 18: Local development setup instructions

End of Report

Submitted to: Bob Besharat, AML-3503 Cloud Computing for Big Data and AI, Lambton College, April 2026.